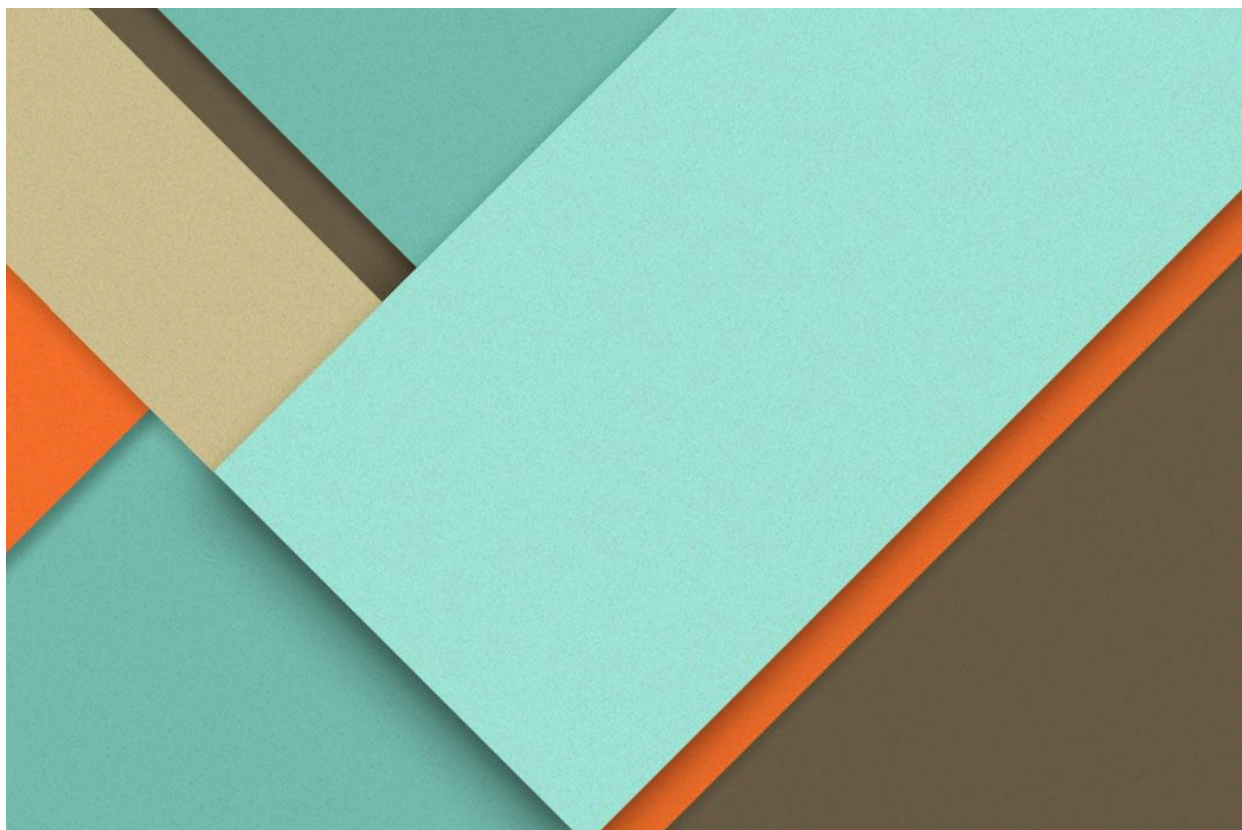




El - Práctica 3

05/06/2026

Pablo Mira Amante

2025-2026

Introducción

En esta práctica realizaremos el último paso para el buscador. Para respetamos 2 puntos clave:

- **Tokenización e indexación:** Hay que respetar la estructura definida, si se puede por temas de tiempo se intentará optimizar el proceso de tokenización e indexación, siempre manteniendo las clases en su estructura original.
- **Gestión de resultados:** Se necesita hacer una cola de prioridad para almacenar los documentos y ordenarlos según su prioridad.
- **"Capturas":** Para poder demostrar que se donde esta cada cosa en el código, como no hay una revisión presencial, se me ha ocurrido insertar los cachos de código en formato texto tal cual como se encontraron en el código siempre que se considere necesario, como en las mejoras realizadas.

Y contamos con 2 hipótesis de partida:

- **Fiabilidad:** Los modelos probabilísticos avanzados ofrecerán resultados precisos en la recuperación de información (Si aplicamos DFR o Okapi BM25 aseguramos un correcto funcionamiento).
- **Estructura de datos:** Si optimizo las estructuras de datos en memoria evitando búsquedas repetitivas se reducirá drásticamente el tiempo de ejecución de las fórmulas. Esta va a ser mi prioridad a la hora de optimizar.

Arquitectura del Buscador

Si al igual que en la anterior práctica definimos su arquitectura en tres capas, nos encontramos con:

Capa de Datos:

- Obtiene su núcleo de una herencia pública directa del indexador.
 - Esto permite que el buscador tenga acceso directo a toda la parte pública del indexador y a la representación en memoria del índice invertido sin necesidad de duplicar los datos de la colección o reconstruirlos.
- Creación de “ResultadosRI”.
 - Funciona como un contenedor de datos para las estadísticas de búsqueda
- Cola de prioridad.
- Caché temporal.

Capa de Lógica:

- El motor de búsqueda, los métodos sobrecargados Buscar().
- Implementación de los algoritmos de búsqueda DFR y Okapi.
- Algoritmo de ordenación con “Unordered_map”

Capa de Soporte:

- Métodos que imprimen por pantalla
- Ciertos Getters y Setters
- Funciones auxiliares como “getDocName()” para parsear cadenas de texto y limpiar rutas de archivos

Algoritmo de Búsqueda

Para una mejor implementación que permita poder gestionarlo de la forma más eficiente (al mismo estilo que en la práctica del indexador), se diferencia entre 2 formas de invocar a la función de búsqueda, una para búsquedas simples y otra para directorios (Baterías de pruebas).

- Búsqueda sencilla

```
bool Buscador::Buscar(const int& numDocumentos) {
```

- Paso 1 - Limpieza y validación inicial.
 - Se vacía la cola de prioridad global y se valida la existencia de condiciones mínimas para la búsqueda.
- Paso 2 - Generación de la caché de acceso directo.
 - En lugar de buscar la información de cada documento dentro del índice durante la evaluación, se crea un vector de punteros idToInfDoc. Este vector actúa como una tabla de mapeo directo $O(1)$.
- Paso 3 - Precomputación a nivel término
 - El algoritmo itera sobre cada término almacenado en la pregunta. Si el término de la pregunta también existe en el corpus, se procede a calcular las variables matemáticas de los modelos que solo dependen del término y no del documento.
- Paso 4 - Evaluación de los documentos (Bucle)
 - Una vez preparadas las variables, se recorre la lista de postings (Los documentos elegidos).
 - Para cada documento se extrae su longitud al instante usando el vector caché "idToInfDoc".
 - Dependiendo del modelo configurado, se aplica la fórmula matemática final evaluando la frecuencia del término en el documento y la penalización por longitud, calculando una puntuación parcial para ese término.
 - Este score se suma en una tabla hash acumulando los puntos que el documento va consiguiendo con cada palabra de la pregunta evaluada.
- Paso 5 - Ordenación de los resultados
 - Una vez procesados todos los términos, el contenido del mapa de puntuaciones se transfiere a un vector resultados y este vector se ordena rápidamente de mayor a menor relevancia utilizando la función "sort" de std.
- Paso 6 - Volcado final
 - se recorta la lista hasta los N primeros y se introducen en la cola de prioridad "docsOrdenados" para que el programa pueda imprimirlos después.

- Búsqueda por directorios

```
bool Buscador::Buscar(const string& dirPreguntas, const int& numDocumentos,  
                     const int& numPregInicio, const int& numPregFin) {
```

- Paso 1 - Limpieza y optimización global.
 - Se vacía la cola de prioridad global, se construye la caché de acceso directo ("idToInfDoc") y se calcula la longitud media del corpus una sola vez al principio.
 - Esto al hacerlo fuera del bucle nos evita el coste de reconstruir la tabla hash y el vector para cada archivo.
- Paso 2 - Bucle iterativo de lectura e indexación dinámica
 - En cada iteración, el método construye dinámicamente la ruta en disco del fichero de la consulta, abre el archivo, vuelca todo su contenido en un string y lo cierra.
 - Invoca al método heredado IndexarPregunta(content) para tokenizar, filtrar y dejar la consulta activa en las estructuras internas de la clase base. Si la pregunta está vacía o carece de términos con contenido, se ignora.
- Paso 3 - Evaluación matemática y acumulación local de puntuaciones
 - Para cada pregunta activa en la iteración actual, se crea un mapa hash local y se repite la misma estrategia que en el método individual.
- Paso 4 - Ordenación local de los resultados
 - Una vez sumadas las puntuaciones de todos los términos de la pregunta actual, se vuelca en un vector para su posterior ordenación, igual que en el método individual.
- Paso 5 - Volcado final
 - se recorta la lista hasta los N primeros y se introducen en la cola de prioridad "docsOrdenados" al que se le asigna el identificador de la pregunta actual (i).
 - Gracias a la sobrecarga del operador de comparación, la cola de prioridad global auto-ordena los elementos con un criterio doble:
 - Primero agrupa por el número de preguntas de forma ascendente
 - En caso de empate los ordena por su valor de similitud de forma descendente.
 - Esto garantiza que al vaciar la cola o imprimir los resultados, la salida quede perfectamente tabulada y formateada por bloques de preguntas de forma nativa.

Problemas encontrados durante la realización

- Problemas a la hora de acceder a la clase indexador.
 - Al haber definido el constructor vacío dentro de la parte privada, no se puede acceder, evitando que se pueda crear desde la clase buscador.
 - La forma de solucionarlo ha sido hacer la clase amiga de Buscador (friend class Buscador;)
 - Esto mismo se aplica para todas aquellas clases dentro de IndexadorInformación.

Alternativas descartadas

- Descarte de inserción directa en la cola:
 - **DESCARTADO:** Si lo insertamos directamente después del cálculo es muy ineficiente, incrementando mucho el tiempo de búsqueda. Esto se debe a que requeriría buscar y actualizar pesos dentro de una estructura en árbol u obligaría a insertar duplicados y filtrarlos después.
- Cambio a PUBLIC o PROTECTED atributos y métodos durante la Herencia del tokenizador e indexador
 - **DESCARTADO:** Como he puesto en los puntos clave, la estructura central no voy a cambiarla hasta este punto, principalmente por los problemas que podría causar.
- Cambio a unordered_map durante la indexación
 - Una de las mejoras implementadas durante la indexación no ayudaba durante la búsqueda.
 - **DESCARTADO:** Por temas de falta de tiempo para el estudio del resto de materias, me resulta difícil refactorizar todo y he preferido aplicar el tiempo a mejoras de la clase buscador.
- Lectura completa de todos los archivos y luego su procesado
 - **DESCARTADO:** Aunque al principio la idea era tentadora, ya en mi cabeza sonaba como que solamente tendría que leer una única vez el disco, ahorrando mucho tiempo, luego ha resultado que no era factible.

Análisis de Mejoras realizadas

- **Uso de “\n”.**
 - Se sigue aplicando desde la primera práctica.
- **Mejora de leaked de memoria en la práctica del indexador.**
 - He pasado valgrind para el estudio de la eficiencia de memoria, y ha saltado con una posible fuga de memoria en la práctica del indexador, para ser exactos todas las clases de “indexadorInformacion”.
 - Esto ocurre al escribir en el fichero binario, donde se realiza la escritura de golpe de la estructura InfDoc y la clase InfColecciones. Esto causa que se introduzcan bytes de relleno no inicializados por el compilador (padding de alineación de 64 bit).
 - Para solucionarlo he inicializado las estructuras utilizando memset en los constructores y operadores de asignación.
- **Uso del aislamiento local (Mejora recomendada por un modelo de IA).**
 - Sirve para asegurar que el separador decimal sea siempre un punto, sin importar el idioma del sistema operativo. En sistemas operativos españoles, escribe siempre una coma en vez de un punto para los decimales.

```
// Forzar el locale classic para usar punto en los decimales
ostream& os = cout;
auto oldLocale = os.imbue(locale::classic());
```

- **Acceso a longitudes de Documento en Tiempo Constante $O(1)$.**
 - Al inicio de cada búsqueda, se construye un vector de punteros de tamaño $n+2$ llamado idToInfDoc (vector `<const InfDoc*>`). Este vector se indexa directamente con el idDoc numérico.

```
// Mapeo rapido de ID de documento a InfDoc
vector<const InfDoc*> idToInfDoc(N + 2, nullptr);
for (auto const& par : indiceDocs) {
    idToInfDoc[par.second.idDoc] = &par.second;
}
```

- Esto reduce la búsqueda de longitud del documento a un acceso directo a memoria indexado ($O(1)$) eliminando por completo las búsquedas asociativas por texto durante el bucle crítico.
- **Precomputación de Factores a Nivel de Término.**
 - Calcular funciones matemáticas como $\log_2$ es pesado computacionalmente si se realiza repetitivamente por cada término y documento.
 - Por tanto, los cálculos que dependen únicamente del término de la consulta (t) se realizan fuera del bucle de documentos.

- Para DFR: Parámetros como logaritmos y el factor de colección se precaculan una sola vez por cada término de la query.
- Para BM25: El valor $IDF(t)$ y el producto $IDF(t) \cdot (k+1)$ se pre calculan antes de iterar por los documentos que contienen el término
- Cálculo:

```
if (formSimilitud == 0) {

    wt_q = (double)parPreg.second.ft / k;

    lambda_t = (double)ft / N;

    log2_1_lambda = log2(1.0 + lambda_t);

    log2_ratio = log2((1.0 + lambda_t) / lambda_t);

    term_factor = (double)(ft + 1) / nt;

} else if (formSimilitud == 1) {

    double num_idf = N - nt + 0.5;

    double den_idf = nt + 0.5;

    idf = log2(num_idf / den_idf);

    idf_times_k1_plus_1 = idf * (k1 + 1.0);

}
```

- Aplicación:

```
double scoreTermino = 0.0;

if (formSimilitud == 0) {

    // Formula DFR

    double f_td_star = ft_d * log2(1.0 + c * avr_ld / ld);

    double w_td = (log2_1_lambda + f_td_star * log2_ratio) * (term_factor /
(f_td_star + 1.0));

    scoreTermino = wt_q * w_td;

} else if (formSimilitud == 1) {

    // Formula BM25

    double K = k1 * (1.0 - b + b * (double)ld / avr_ld);

    scoreTermino = idf_times_k1_plus_1 * ft_d / (ft_d + K);

}
```

- Variación al archivo main para poder lanzar los diferentes corpus:
 - Adjuntado junto con el resto en /src
- Sobrecarga del operador comparación en buscador.h.
 - Su utilidad ya se ha explicado anteriormente

Análisis de mejora con respecto a la primera versión

Complejidad temporal

Estos son los resultados de que el main recorra la totalidad del corpus documental.

- **Versión final**
 - Tiempo total de ejecución: Ha tardado 0.100719 segundos.

Complejidad Espacial

- **Versión final**
 - total heap usage: 523,106 allocs, 523,106 frees, 55,701,660 bytes allocated

También añadimos los resultados de las cuatro pruebas trec_*_*stem con la correspondiente gráfica:

- **BM25_nostem:**

Queryid (Num): All

Total number of documents over all queries

Retrieved: 16219

Relevant: 324

Rel_ret: 298

Interpolated Recall - Precision Averages:

at 0.00 0.7094

at 0.10 0.7094

at 0.20 0.7051

at 0.30 0.6763

at 0.40 0.6469

at 0.50 0.6275

at 0.60 0.5573

at 0.70 0.5303

at 0.80 0.5185

at 0.90 0.4429

at 1.00 0.4385

Average precision (non-interpolated) for all rel docs(averaged over queries) -> 0.5827

- **BM25_stem:**

Queryid (Num): All

Total number of documents over all queries

Retrieved: 20647

Relevant: 324

Rel_ret: 306

Interpolated Recall - Precision Averages:

at 0.00 0.7330

at 0.10 0.7330

at 0.20 0.7310

at 0.30 0.7127

at 0.40 0.6883

at 0.50 0.6753

at 0.60 0.5782

at 0.70 0.5485

at 0.80 0.5370

at 0.90 0.4657

at 1.00 0.4581

Average precision (non-interpolated) for all rel docs(averaged over queries)

0.6089

- DFR_nostem:

Queryid (Num): All

Total number of documents over all queries

Retrieved: 16219

Relevant: 324

Rel_ret: 298

Interpolated Recall - Precision Averages:

at 0.00 0.7301

at 0.10 0.7301

at 0.20 0.7141

at 0.30 0.6935

at 0.40 0.6682

at 0.50 0.6507

at 0.60 0.5786

at 0.70 0.5594

at 0.80 0.5434

at 0.90 0.4747

at 1.00 0.4654

Average precision (non-interpolated) for all rel docs(averaged over queries)

0.6039

- DFR_stem:

Queryid (Num): All

Total number of documents over all queries

Retrieved: 20647

Relevant: 324

Rel_ret: 306

Interpolated Recall - Precision Averages:

at 0.00 0.7591

at 0.10 0.7547

at 0.20 0.7348

at 0.30 0.7076

at 0.40 0.6791

at 0.50 0.6667

at 0.60 0.5828

at 0.70 0.5675

at 0.80 0.5535

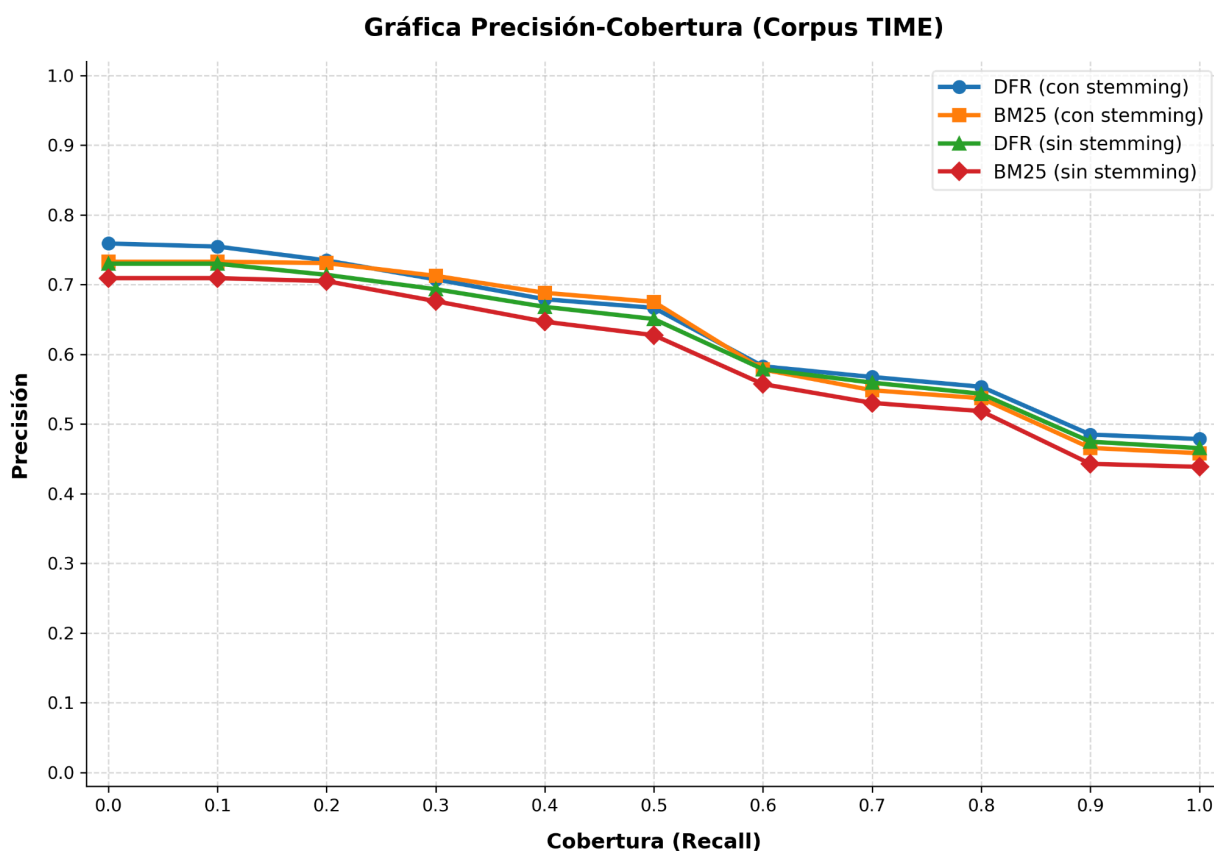
at 0.90 0.4849

at 1.00 0.4785

Average precision (non-interpolated) for all rel docs(averaged over queries)

0.6170

Y la gráfica:



Conclusión

Este es el final de las prácticas, creo que ha quedado algo bastante interesante y me gustaría acabar de desarrollarlo con más tiempo para aplicarlo en algún proyecto que pueda hacer este verano. Como por ejemplo un buscador de archivos más eficiente que el de Windows, que no es muy difícil. Feliz Verano profe :).